Up or Down? Adaptive Rounding for Post-Training Quantization

26/03/23

AdaRound (PTQ)

<u>Adaptive Rounding => Scale은 사전에 fixed 되어 있다고 가정</u>

[기존 문제]

On the other hand, much attention has recently been dedicated to *post-training quantization* methods (Nagel et al., 2019; Cai et al., 2020; Choukroun et al., 2019; Banner et al., 2019), which can be more easily applied in practice. These types of methods allow for network quantization to happen on-the-fly when deploying models, without the user of the model spending time and energy on quantization. Our work focuses on this type of network quantization.

Rounding-to-nearest is the predominant approach for all neural network weight quantization work that came out thus far. This means that the weight vector w is rounded to the nearest representable quantization grid value in a fixed-point grid by

$$\widehat{\mathbf{w}} = \mathrm{s} \cdot clip\left(\left\lfloor \frac{\mathbf{w}}{\mathrm{s}} \right\rceil, \mathrm{n}, \mathrm{p}\right), \tag{1}$$

where s denotes the quantization scale parameter and, n and p denote the negative and positive integer thresholds for clipping. We could round any weight down by replacing $\lfloor \cdot \rceil$ with $\lfloor \cdot \rfloor$, or up using $\lceil \cdot \rceil$. But, rounding-to-nearest seems the most sensible, as it minimizes the difference per-weight in the weight matrix. Perhaps surprisingly, we show that for post-training quantization, rounding-to-nearest is not optimal.

1. 항상 clip을 반올림 방식의 rounding-to-nearest를 사용

이게 과연 최선인가?

[Motivation]

To gain an intuitive understanding for why rounding-to-nearest may not be optimal, let's look at what happens when we perturb the weights of a pretrained model. Consider a neural network parametrized by the (flattened) weights $\mathbf{w}$. Let $\Delta\mathbf{w}$ denote a small perturbation and $\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{w})$ denote the task loss that we want to minimize. Then

$$\mathbb{E}\left[\mathcal{L}\left(\mathbf{x}, \mathbf{y}, \mathbf{w} + \Delta\mathbf{w}\right) - \mathcal{L}\left(\mathbf{x}, \mathbf{y}, \mathbf{w}\right)\right] \tag{2}$$

$$\overset{(a)}{\approx} \mathbb{E}\left[\Delta\mathbf{w}^T \cdot \nabla_{\mathbf{w}}\mathcal{L}\left(\mathbf{x}, \mathbf{y}, \mathbf{w}\right) + \frac{1}{2}\Delta\mathbf{w}^T \cdot \nabla^2_{\mathbf{w}}\mathcal{L}\left(\mathbf{x}, \mathbf{y}, \mathbf{w}\right) \cdot \Delta\mathbf{w}\right] \tag{3}$$

$$= \Delta\mathbf{w}^T \cdot \mathbf{g}^{(\mathbf{w})} + \frac{1}{2}\Delta\mathbf{w}^T \cdot \mathbf{H}^{(\mathbf{w})} \cdot \Delta\mathbf{w}, \tag{4}$$

where (a) uses the second order Taylor series expansion. $\mathbf{g}^{(\mathbf{w})}$ and $\mathbf{H}^{(\mathbf{w})}$ denote the expected gradient and Hessian of the task loss $\mathcal{L}$ w.r.t. $\mathbf{w}$, i.e.,

$$\mathbf{g}^{(\mathbf{w})} = \mathbb{E}\left[\nabla_{\mathbf{w}}\mathcal{L}\left(\mathbf{x}, \mathbf{y}, \mathbf{w}\right)\right] \tag{5}$$

$$\mathbf{H}^{(\mathbf{w})} = \mathbb{E}\left[\nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{w})\right]. \tag{6}$$

$$\mathbf{H}^{(\mathbf{w})} = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}, \tag{7}$$

then the increase in task loss due to the perturbation is (approximately) proportional to

$$\Delta\mathbf{w}^T \cdot \mathbf{H}^{(\mathbf{w})} \cdot \Delta\mathbf{w} = \Delta\mathbf{w}_1^2 + \Delta\mathbf{w}_2^2 + \Delta\mathbf{w}_1\Delta\mathbf{w}_2. \tag{8}$$

For the terms corresponding to the diagonal entries $\Delta\mathbf{w}_1^2$ and $\Delta\mathbf{w}_2^2$, only the magnitude of the perturbations matters. Hence rounding-to-nearest is optimal when we only consider these diagonal terms in this example. However, for the terms corresponding to the $\Delta\mathbf{w}_1\Delta\mathbf{w}_2$, the sign of the perturbation matters, where opposite signs of the two perturbations improve the loss. To minimize the overall impact of quantization on the task loss, we need to trade-off between the contribution of the diagonal terms and the off-diagonal terms. Rounding-to-nearest ignores the off-diagonal contributions, making it often sub-optimal.

g=0, wHw만 고려.

Nearest-Rounding의 경우, w^2은 최소화하지만, w1w2의 term은 최소화하지 못함

$$\Delta \mathbf{w}^T \cdot \mathbf{H}^{(\mathbf{w})} \cdot \Delta \mathbf{w} = \Delta \mathbf{w}_1^2 + \Delta \mathbf{w}_2^2 + \Delta \mathbf{w}_1 \Delta \mathbf{w}_2.$$

즉, red에서는 optimal하지만, blue에서는 off-diagnol 즉, 비대각에서는 optimal X.

서로 부호가 반대일 경우, 음수가 되어 Quantization Error를 더 상쇄시킴.

즉, 기존의 nearest-rounding은 단순 가까운쪽으로 반올림하기 때문에, 이러한 부호 조합을 통한 손실 최적화 X.

따라서, Loss를 최소화하는 w를 adaptive하게 round하여 찾자!

When quantizing a pretrained NN, our aim is to minimize the performance loss incurred due to quantization. Assuming per-layer weight quantization[1], the quantized weight $\widehat{\mathbf{w}}_i^{(\ell)}$ is

$$\widehat{\mathbf{w}}_i^{(\ell)} \in \left\{ \mathbf{w}_i^{(\ell),floor}, \mathbf{w}_i^{(\ell),ceil} \right\}, \tag{9}$$

where

$$\mathbf{w}_i^{(\ell),floor} = \mathbf{s}^{(\ell)} \cdot clip\left(\left\lfloor \frac{\mathbf{w}_i^{(\ell)}}{\mathbf{s}^{(\ell)}} \right\rfloor, \mathrm{n}, \mathrm{p} \right) \tag{10}$$

$$\underset{\Delta \mathbf{w}}{\arg\min} \quad \mathbb{E}\left[\mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{w} + \Delta \mathbf{w}) - \mathcal{L}(\mathbf{x}, \mathbf{y}, \mathbf{w})\right] \tag{11}$$

$$\underset{\Delta \mathbf{w}^{(\ell)}}{\arg\min} \quad \mathbb{E}\left[\mathbf{g}^{(\mathbf{w}^{(\ell)})^T} \Delta \mathbf{w}^{(\ell)} + \frac{1}{2} \Delta \mathbf{w}^{(\ell)^T} \mathbf{H}^{(\mathbf{w}^{(\ell)})} \Delta \mathbf{w}^{(\ell)} \right]. \tag{12}$$

$$\underset{\Delta \mathbf{w}^{(\ell)}}{\arg\min} \quad \mathbb{E}\left[\Delta \mathbf{w}^{(\ell)^T} \mathbf{H}^{(\mathbf{w}^{(\ell)})} \Delta \mathbf{w}^{(\ell)} \right]. \tag{13}$$

이때 delta_w는 binary 즉, 올릴지 내릴지에 대한 binary variables로 QUBO 문제로 변환

하지만, wHw를 최적화하는 것은 아래의 두 가지 문제가 존재.

1. $\mathbf{H}^{(\mathbf{w}^{(\ell)})}$ suffers from both computational as well memory complexity issues even for moderately sized layers.
2. (13) is an NP-hard optimization problem. The complexity of solving it scales rapidly with the dimension of $\Delta \mathbf{w}^{(\ell)}$, again prohibiting the application of (13) to even moderately sized layers (Kochenberger et al., 2014).

1. Hessian 연산량 heavy
2. QUBO의 NP-hard O(N^2) 시간 복잡도

1. Hessian to MSE

3.2. From Taylor expansion to local loss

To understand the cause of the complexity associated with $\mathbf{H}^{(\mathbf{w}^{(\ell)})}$, let us look at its' elements. For two weights in the same fully connected layer we have

$$\frac{\partial^2 \mathcal{L}}{\partial \mathbf{W}_{i,j}^{(\ell)} \partial \mathbf{W}_{m,o}^{(\ell)}} = \frac{\partial}{\partial \mathbf{W}_{m,o}^{(\ell)}} \left[\frac{\partial \mathcal{L}}{\partial \mathbf{z}_i^{(\ell)}} \cdot \mathbf{x}_j^{(\ell-1)} \right] \tag{14}$$

$$= \frac{\partial^2 \mathcal{L}}{\partial \mathbf{z}_i^{(\ell)} \partial \mathbf{z}_m^{(\ell)}} \cdot \mathbf{x}_j^{(\ell-1)} \mathbf{x}_o^{(\ell-1)}, \tag{15}$$

where $\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{x}^{(\ell-1)}$ are the preactivations for layer ℓ and $\mathbf{x}^{(\ell-1)}$ denotes the input to layer ℓ. Writing this in matrix formulation (for flattened $\mathbf{w}^{(\ell)}$), we have (Botev et al., 2017)

$$\mathbf{H}^{(\mathbf{w}^{(\ell)})} = \mathbb{E}\left[\mathbf{x}^{(\ell-1)} \mathbf{x}^{(\ell-1)^T} \otimes \nabla^2_{\mathbf{z}^{(\ell)}} \mathcal{L} \right], \tag{16}$$

Chain rule에 따라, hessian을 위와 같이 분해.

하지만, hessian 연산은

d_in x d_in * (d_out x d_out) => (d_in x d_in) * (d_out x d_out) [*: Kronecker product]

⇨ (d_in x d_out) x (d_in x d_out)

너무 연산량이 큼. 따라서, L을 diagonal이라 가정하고 진행

1. 역전파(Backpropagation) 과정의 제거

원래의 Hessian 행렬 $H(w^{(l)})$을 계산하려면 네트워크의 후속 레이어들을 통해 **2차 미분계수($\nabla^2_{z^{(l)}} L$)를 역전파**해야 합니다 [1]. 이는 계산량과 메모리 측면에서 매우 큰 부담이 되어 중간 크기의 레이어에도 적용하기 불가능(prohibiting)할 정도입니다 [2]. 하지만 이를 **상수 대각 행렬**로 가정함으로써, 더 이상 후속 레이어나 태스크 손실에 대한 정보를 알 필요 없이 해당 레이어의 정보만으로 최적화가 가능해졌습니다 [3].

where $\otimes$ denotes Kronecker product of two matrices and $\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}$ is the Hessian of the task loss w.r.t. $\mathbf{z}^{(\ell)}$. It is clear from (16) that the complexity issues are mainly caused by $\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}$ that requires backpropagation of second derivatives through the subsequent layers of the network. To tackle this, we make the assumption that the Hessian of the task loss w.r.t. the preactivations, i.e., $\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}$ is a diagonal matrix, denoted by $diag\left(\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}_{i,i}\right)$. This leads to

$$\mathbf{H}^{(\mathbf{w}^{(\ell)})} = \mathbb{E}\left[\mathbf{x}^{(\ell-1)}\mathbf{x}^{(\ell-1)^T} \otimes diag(\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}_{i,i})\right]. \quad (17)$$

Note that the approximation of $\mathbf{H}^{(\mathbf{w}^{(\ell)})}$ expressed in (17) is not diagonal. Plugging (17) into our equation for finding the rounding vector that optimizes the loss (13), we obtain

$$\underset{\Delta\mathbf{W}^{(\ell)}_{k,:}}{\arg\min} \quad \mathbb{E}\left[\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}_{k,k} \cdot \Delta\mathbf{W}^{(\ell)}_{k,:}\mathbf{x}^{(\ell-1)}\mathbf{x}^{(\ell-1)^T}\Delta\mathbf{W}^{(\ell)^T}_{k,:}\right] \quad (18)$$

$$\overset{(a)}{=} \underset{\Delta\mathbf{W}^{(\ell)}_{k,:}}{\arg\min} \quad \Delta\mathbf{W}^{(\ell)}_{k,:}\mathbb{E}\left[\mathbf{x}^{(\ell-1)}\mathbf{x}^{(\ell-1)^T}\right]\Delta\mathbf{W}^{(\ell)^T}_{k,:} \quad (19)$$

$$= \underset{\Delta\mathbf{W}^{(\ell)}_{k,:}}{\arg\min} \quad \mathbb{E}\left[\left(\Delta\mathbf{W}^{(\ell)}_{k,:}\mathbf{x}^{(\ell-1)}\right)^2\right], \quad (20)$$

where the optimization problem in (13) now decomposes into independent sub-problems in (18). Each sub-problem deals with a single row $\Delta\mathbf{W}^{(\ell)}_{k,:}$ and (a) is the outcome of making a further assumption that $\nabla^2_{\mathbf{z}^{(\ell)}}\mathcal{L}_{i,i} = c_i$ is a constant independent of the input data samples. It is worthwhile to note that optimizing (20) requires no knowledge of the subsequent layers and the task loss. In (20), we are simply minimizing the Mean Squared Error (MSE) introduced in the preactivations $\mathbf{z}^{(\ell)}$ due to quantization. This is the same

Diagonalize를 통해, L을 상수화. ***(즉, 현재 layer에서만 최적화하겠다!)***

이를 통해 MSE로 최적화. 이를 통해, Hessian의 계산복잡도 문제 해결.

하지만, 여전히 QUBO의 NP-Hard 문제 존재.

⇨ Continuous Soft optimization 문제로 설정.

$$\arg\min_{\mathbf{V}} \left\| \mathbf{W}\mathbf{x} - \widetilde{\mathbf{W}}\mathbf{x} \right\|_F^2 + \lambda f_{reg}(\mathbf{V}), \qquad (21)$$

where $\|\cdot\|_F^2$ denotes the Frobenius norm and $\widetilde{\mathbf{W}}$ are the soft-quantized weights that we optimize over

$$\widetilde{\mathbf{W}} = \mathrm{s} \cdot clip\left(\left\lfloor \frac{\mathbf{W}}{\mathrm{s}} \right\rfloor + h(\mathbf{V}), \mathrm{n}, \mathrm{p} \right). \qquad (22)$$

h(V): 올림/내림을 결정 offset

연속적인 학습이 binary 변수 학습과 어떤 차이가 있어서 효율적인거야?

연속적인 학습(연속적 완화, Continuous Relaxation)이 이진(Binary) 변수 학습보다 효율적인 이유는 **수학적 복잡도를 획기적으로 낮추어 현대적인 최적화 도구를 사용할 수 있게 해주기 때문**입니다.

소스의 내용을 바탕으로 그 구체적인 차이점과 효율성의 원인을 정리해 드립니다.

1. NP-hard 문제의 회피

- **이진 변수 학습의 한계:** 각 가중치를 위로 올릴지 아래로 내릴지 결정하는 문제는 **이차 비제약 이진 최적화(QUBO)** 문제로 분류됩니다 [1]. 이는 **NP-hard** 문제로, 변수의 수가 조금만 늘어나도 가능한 조합이 기하급수적으로 많아져 계산이 사실상 불가능해집니다 [2] [3].
- **연속적 완화의 해법:** AdaRound는 이 이산적인 문제를 0과 1 사이의 연속적인 값을 가질 수 있는 **연속적 최적화 문제로 변환(Relaxation)**합니다 [3] [4]. 이를 통해 "모든 경우의 수를 따져보는" 방식에서 벗어날 수 있습니다.

2. 미분 가능성 및 경사 하강법(SGD) 사용

- **이진 변수:** 0 아니면 1인 이산 변수는 미분이 불가능합니다. 따라서 기울기(Gradient)를 구할 수 없어 효율적인 최적화 알고리즘을 적용하기 어렵습니다.
- **연속 변수 V:** AdaRound는 연속 변수 V와 미분 가능한 함수 $h(V)$(수정된 시그모이드)를 도입합니다 [4] [5]. 함수가 미분 가능해지면 **확률적 경사 하강법(SGD)**이나 **Adam**과 같은 강력하고 빠른 최적화 도구를 사용하여 정답을 효율적으로 찾아갈 수 있습니다 [6] [7].

3. 압도적인 연산 속도 차이

- 소스에 따르면, 이진 문제를 직접 풀려고 시도(Cross-entropy method 등 사용)했을 때는 **수 시간**이 걸렸습니다 [8].
- 반면, 연속적 완화 기법을 적용한 AdaRound는 이를 **단 몇 분**으로 단축시켰습니다 [8]. 실제로 ResNet18 모델 전체를 최적화하는 데 단 **10분**밖에 걸리지 않는 수준입니다 [7] [9].

4. 학습의 안정성과 수렴성

- **안정적 탐색:** 연속적인 공간에서는 학습 초기에는 V가 자유롭게 움직이며 MSE를 줄이는 최적의 방향을 탐색할 수 있습니다 [10].
- **이진화 유도:** 학습 후반부에는 정규화 항(f_{reg})을 통해 이 연속적인 값들을 다시 0 또는 1로 수렴하게 만듭니다 [4] [10]. 즉, **학습은 효율적인 연속 공간에서 하고, 결과는 우리가 원하는 이진 형태**로 얻어내는 영리한 방식입니다 [11].

요약하자면, 연속적 학습은 **"풀 수 없는 이산 문제"를 "미분 가능한 연속 문제"로 바꾸어줌으로써**, 수천만 개의 가중치 조합 사이에서 헤매지 않고 **기울기를 따라 최단 경로로 최적의 반올림 방향을 찾게 해주기 때문에** 훨씬 효율적입니다 [8] [12].

즉, 연속적인 미분 문제로 이를 해결하여, 모든 경우의 수를 따지는 문제를 해결!

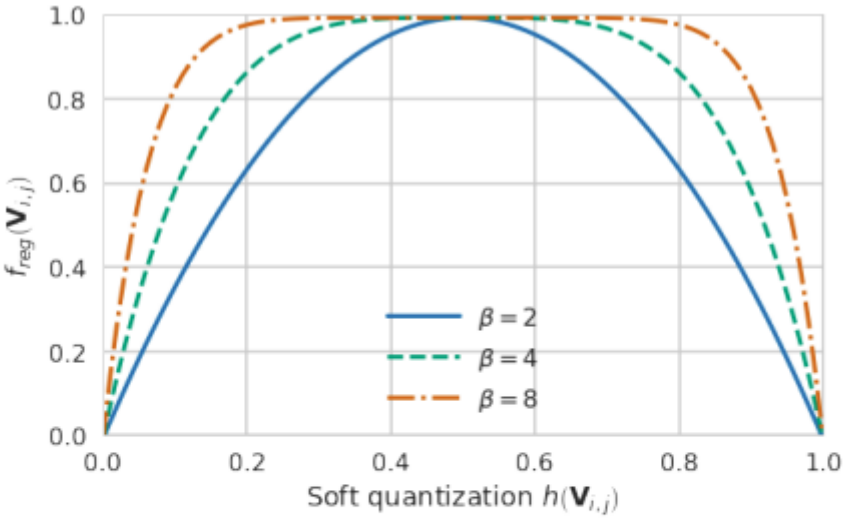

Figure 2. Effect of annealing b on regularization term (24).

encourage $h(\mathbf{V}_{i,j})$ to move to the extremities. For regularization we use

$$f_{reg}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^{\beta}, \qquad (24)$$

Regularization factor는 h(V)가 0, 1로 수렴하도록 유도하며,

B가 작을수록 급하기 때문에, 끝으로 갈수록 B를 줄이고 초기에는 완만하게 B를 크게 설정하여 아무데나 갈 수 있도록 유도

즉, 기존의 0,1 값을 무조건 가지는 w를 연속적인 실수 값을 가지는 V로 설정하여 NP-hard 문제를 해결.

Asymmetric reconstruction formulation

결과적으로 AdaRound는 **계산은 현재 레이어에만 집중(상수 가정)하면서도, 실제 오차 흐름은 이전 레이어부터의 누적치를 반영하여 최적화**를 수행.

$$\arg\min_{\mathbf{V}} \left\| f_a(\mathbf{W}\mathbf{x}) - f_a\left(\widetilde{\mathbf{W}}\hat{\mathbf{x}}\right) \right\|_F^2 + \lambda f_{reg}(\mathbf{V}), \quad (25)$$

1. Wx가 의미 있는 이유: 이상적인 기준점

- Wx는 양자화되지 않은 원래의 전정밀도(FP32) 모델이 깨끗한 입력 x를 받았을 때 내뱉는 **가장 정확한 출력값**입니다 [1].
- 우리의 최종 목표는 양자화된 모델이 원래 모델의 동작을 그대로 흉내 내는 것이므로, **비교의 기준(Target)은 항상 오차가 없는 Wx**여야 합니다 [1].

2. 왜 $\tilde{W}x$가 아니라 $\tilde{W}\hat{x}$로 계산해야 하는가?

- **오차 누적의 현실:** 실제 양자화된 모델이 구동될 때, 현재 레이어는 깨끗한 x를 받는 것이 아니라 앞선 레이어들에서 이미 양자화 오차가 섞여 들어온 **$\hat{x}$**를 입력으로 받게 됩니다 [1] [2].
- **보정(Correction) 메커니즘:** 만약 단순히 Wx와 $\tilde{W}x$를 비교하여 최적화한다면, 이전 레이어에서 발생한 오차는 무시한 채 현재 레이어의 가중치만 원본과 비슷하게 맞추게 됩니다.
- 하지만 **비대칭 구성(Wx vs $\tilde{W}\hat{x}$)**을 사용하면, "비록 입력값($\hat{x}$)에 오차가 섞여 들어왔을지라도, 현재 레이어의 반올림($\tilde{W}$)을 잘 조절해서 **최종 출력만큼은 원래 모델의 깨끗한 출력(Wx)과 같아지도록** 만들어라"는 명령이 가능해집니다 [1] [2].

3. "원래는 quantized x로 해야 하는 것 아니냐"에 대한 답변

- 맞습니다. 만약 우리가 '입력과 출력의 관계'를 단순히 유지하고 싶다면 $W\hat{x}$와 $\tilde{W}\hat{x}$를 비교했을 것입니다.
- 하지만 소스에 따르면, 그렇게 할 경우 이전 레이어에서 발생한 오차가 다음 레이어로 그대로 전달되고 증폭되어 **오차 누적(Error accumulation)** 현상이 발생합니다 [2].
- **비대칭 방식**은 이 누적된 오차를 현재 레이어의 반올림 결정으로 **상쇄(Counteract)**할 수 있는 기회를 제공하기 때문에, 결과적으로 모델 전체의 정확도를 복구하는 데 훨씬 유리합니다 [1] [...].

요약하자면: 비대칭 구성은 **"이상적인 목표(Wx)"를 바라보되, "오차가 섞인 현실($\hat{x}$)"을 입력으로 넣어 최적화**함으로써, 현재 레이어가 앞선 레이어들의 실수까지 만회하도록 만드는 매우 영리한 전략입니다 [1] [4]. 이 방식을 통해 AdaRound는 4비트와 같은 낮은 비트수에서도 높은 정확도를 유지할 수 있습니다